

Universidad del Valle de Guatemala

Construcción de Compiladores – CC-3032, sección 10

Guion de exposición en video

Laboratorio 01

Uso de generadores de analizadores léxicos y sintácticos

Analizador de Compiscript

Grupo: DragonsSlayers 2.0

Integrante 1: Hugo Daniel Barillas Ajín – 23556

Integrante 2: Ernesto Ascencio – 23009

Duración objetivo: 9 minutos 40 segundos

Restricción: máximo 10 minutos

Guatemala, 2026

Cómo utilizar este guion

Este guion corresponde a una exposición continua de 9 minutos 40 segundos, distribuida únicamente entre Hugo y Ernesto. Cada escena indica el tiempo objetivo, la acción que debe mostrarse y un texto que puede leerse casi literalmente. Las cajas azules no se leen; describen la evidencia visual.

Los tiempos dejan 20 segundos de margen respecto del máximo. Los ocho archivos deben estar abiertos o accesibles antes de grabar. No se debe afirmar que Compiscript se ejecuta o se compila a código objeto: el alcance es exclusivamente léxico y sintáctico.

Índice

Cómo utilizar este guion	i
1 Preparación antes de grabar	1
1.1 Ventanas y archivos	1
1.2 Material obligatorio	1
2 Distribución definitiva del tiempo	1
3 Guion por escenas	2
Escena 1. Presentación – 00:00–00:35	2
Escena 2. Fundamento teórico – 00:35–01:15	2
Escena 3. ANTLR, arquitectura y alcance – 01:15–01:55	2
Escena 4. Interfaz y criterios de complejidad – 01:55–02:30	3
Escena 5. Complejidad baja válida – 02:30–03:00	3
Escena 6. Errores en complejidad baja – 03:00–04:00	3
Escena 7. Complejidad media válida – 04:00–04:40	3
Escena 8. Errores en complejidad media – 04:40–05:50	4
Escena 9. Recuperación y diagnósticos – 05:50–06:40	4
Escena 10. Tokens, árbol y exportaciones – 06:40–07:20	4
Escena 11. CLI y pruebas – 07:20–08:00	4
Escena 12. Ejecución y escritorio – 08:00–08:40	4
Escena 13. Comprobación de la rúbrica – 08:40–09:15	5
Escena 14. Conclusiones – 09:15–09:40	5
4 Referencia para la demostración	5
4.1 Matriz oficial de la rúbrica	5
4.2 Correspondencia de complejidad	6

4.3	Comandos rápidos	6
4.4	Carpeta complementaria de recuperación	6
5	Lista final de verificación	7

1 Preparación antes de grabar

1.1 Ventanas y archivos

1. Aplicación web abierta en <http://localhost:3000>.
2. Editor con `Compiscript.g4`, `analyze.ts` y `antlrErrors.ts`.
3. Terminal ubicada en la raíz del repositorio.
4. Explorador en `examples/rubric/low` y `examples/rubric/medium`.
5. Opcional: aplicación portable de Electron ya construida.

Ejecutar antes de grabar:

```
npm install
npm run generate
npm run check
npm test
npm run dev
```

La suite debe mostrar **3 archivos aprobados y 24 pruebas aprobadas**. Aumentar el tamaño de letra del navegador y de la terminal para que tokens, líneas, columnas y mensajes sean legibles.

1.2 Material obligatorio

- Los ocho casos de `examples/rubric`.
- Los resúmenes 0/0, 3/0, 0/3 y 2/2 para baja y media.
- Al menos un diagnóstico con fase, línea, columna, símbolo y mensaje.
- Tabla de tokens y árbol de parseo.
- Carga de archivos `.cps`, exportaciones, CLI y pruebas.
- Menció breve de ANTLR, Vite, React y Electron.

2 Distribución definitiva del tiempo

Tiempo	Tema	Responsable
00:00–00:35	Presentación y objetivo	Hugo
00:35–01:15	Teoría léxica y sintáctica	Ernesto
01:15–01:55	ANTLR, arquitectura y alcance	Hugo
01:55–02:30	Interfaz y definición de complejidades	Hugo
02:30–03:00	Caso válido de complejidad baja	Hugo

Tiempo	Tema	Responsable
03:00–04:00	Tres casos con errores de complejidad baja	Hugo
04:00–04:40	Caso válido de complejidad media	Ernesto
04:40–05:50	Tres casos con errores de complejidad media	Ernesto
05:50–06:40	Recuperación, ubicación y mensajes	Ernesto
06:40–07:20	Tokens, árbol y exportaciones	Hugo
07:20–08:00	CLI y 24 pruebas automatizadas	Ernesto
08:00–08:40	Ejecución web y aplicación de escritorio	Hugo
08:40–09:15	Comprobación de la rúbrica	Ernesto y Hugo
09:15–09:40	Conclusiones y cierre	Ernesto y Hugo

Nota de grabación: No agregar improvisaciones sin recortar otra escena. Mostrar cada resumen antes de cambiar de archivo, pero no recorrer completa la tabla de tokens ocho veces.

3 Guion por escenas

Escena 1. Presentación

00:00–00:35

Responsable: Hugo

En pantalla: Mostrar portada con nombre del laboratorio, ANTLR 4, TypeScript, Hugo y Ernesto.

Hugo: Buenos días. Somos Hugo Daniel Barillas Ajín y Ernesto Ascencio, del grupo DragonsSlayers 2.0. Presentamos un analizador léxico y sintáctico para archivos Compiscript con extensión punto cps. El objetivo es reconocer tokens, validar la estructura del programa y reportar múltiples errores comprensibles sin detenerse en el primero.

Escena 2. Fundamento teórico

00:35–01:15

Responsable: Ernesto

En pantalla: Mostrar un diagrama breve: caracteres, lexer, tokens, parser y árbol.

Ernesto: El lexer transforma caracteres en tokens como palabras reservadas, identificadores, literales y operadores. Un símbolo que no pertenece al vocabulario produce un error léxico. El parser consume esos tokens y verifica las producciones de la gramática; por ejemplo, una declaración debe terminar con punto y coma. Una omisión estructural es un error sintáctico. El resultado correcto del parser es un árbol de parseo que inicia en program y alcanza EOF.

Escena 3. ANTLR, arquitectura y alcance

01:15–01:55

Responsable: Hugo

En pantalla: Mostrar `Compiscript.g4`, `src/generated` y un diagrama con `analyzeInput` conectado a React, CLI y Electron.

Hugo: ANTLR 4 genera el lexer, parser, listener y visitor TypeScript desde Compiscript punto g cuatro. La gramática cubre tipos, arreglos, funciones, clases, expresiones, condicionales y ciclos. El núcleo analyzeInput es compartido por la web, la consola y Electron, por lo que todas las vistas usan el mismo resultado. El alcance termina en análisis léxico y sintáctico; no se ejecuta Compiscript ni se realiza comprobación semántica.

Escena 4. Interfaz y criterios de complejidad

01:55–02:30

Responsable: Hugo

En pantalla: Mostrar la interfaz y las carpetas de complejidad baja y media.

Hugo: La interfaz permite análisis completo, solo lexer, edición y carga de archivos punto cps. La rúbrica pide cuatro casos bajos y cuatro medios. Todo caso bajo conserva tres tipos de variables, una constante, dos operadores aritméticos, un if, un while y un foreach. Los casos medios agregan un arreglo usado, dos clases, dos objetos, dos funciones y dos llamadas.

Escena 5. Complejidad baja válida

02:30–03:00

Responsable: Hugo

En pantalla: Cargar `examples/rubric/low/valid.cps`, señalar brevemente sus construcciones y analizarlo.

Hugo: El primer archivo cumple toda la complejidad baja. El analizador recorre 112 tokens y lo acepta con cero errores léxicos y cero sintácticos. Esto demuestra el primer criterio y permite observar el árbol completo.

Escena 6. Errores en complejidad baja

03:00–04:00

Responsable: Hugo

En pantalla: Cargar los otros tres archivos de low. Dejar visible cada resumen: 3/0, 0/3 y 2/2.

Hugo: El segundo archivo conserva la misma complejidad e introduce arroba, numeral y dólar: reporta tres errores léxicos y ningún error sintáctico. El tercero omite tres puntos y coma: reporta cero errores léxicos y tres sintácticos. El cuarto combina dos símbolos inválidos con dos omisiones y produce exactamente dos errores de cada fase. En todos los casos el procesamiento continúa hasta las sentencias finales.

Escena 7. Complejidad media válida

04:00–04:40

Responsable: Ernesto

En pantalla: Cargar `examples/rubric/medium/valid.cps`; señalar el arreglo, las clases Estudiante y Profesor, las dos instancias, las funciones y sus llamadas.

Ernesto: El caso medio mantiene todos los requisitos bajos y agrega un arreglo de notas utilizado por for y foreach, dos clases, dos objetos creados con new, las funciones sumar y mostrarCurso, y sus llamadas. Se reconocen 263 tokens y el archivo es aceptado con cero errores en ambas fases.

Escena 8. Errores en complejidad media

04:40–05:50

Responsable: Ernesto

En pantalla: Cargar los otros tres archivos de `medium`. Mostrar en cada resultado los conteos 3/0, 0/3 y 2/2.

Ernesto: El caso léxico medio contiene tres símbolos no reconocidos y conserva cero errores del parser. El caso sintáctico omite tres puntos y coma y conserva cero errores del lexer. El caso mixto reporta dos errores léxicos y dos sintácticos. Las construcciones de complejidad media permanecen en los tres archivos, de modo que no son ejemplos mínimos desconectados de la rúbrica.

Escena 9. Recuperación y diagnósticos

05:50–06:40

Responsable: Ernesto

En pantalla: Ampliar dos errores del caso mixto y señalar fase, línea, columna, símbolo y mensaje. Mostrar `tokenStream.fill()` y `DefaultErrorStrategy`.

Ernesto: Fill obliga al lexer a recorrer la entrada completa. Después, `DefaultErrorStrategy` permite al parser insertar o descartar tokens conceptualmente y sincronizarse para continuar. Cada diagnóstico indica si proviene del lexer o parser, la línea y columna base uno, el símbolo relacionado y una explicación en español. Además se agrupan caracteres inválidos contiguos, se eliminan duplicados y se filtran consecuencias inmediatas para evitar cascadas repetidas.

Escena 10. Tokens, árbol y exportaciones

06:40–07:20

Responsable: Hugo

En pantalla: Volver al caso medio válido. Mostrar tabla de tokens, árbol visual y textual, y los botones de descarga.

Hugo: La tabla conserva índice, tipo, lexema, línea, columna y canal de cada token. El árbol puede explorarse visualmente o descargarse como texto. También se exportan la gramática, la entrada, tokens CSV o JSON y el resultado completo. Todo se genera localmente en el navegador, sin enviar el código a un servidor.

Escena 11. CLI y pruebas

07:20–08:00

Responsable: Ernesto

En pantalla: Ejecutar `npm test` y mostrar 3 archivos y 24 pruebas aprobadas. Luego mostrar un comando CLI de la carpeta `rubric`.

Ernesto: Vitest ejecuta 24 pruebas. Ocho leen directamente la matriz de la rúbrica y verifican aceptación, conteos y construcciones de complejidad. La CLI reutiliza el mismo motor: un archivo válido termina con código cero y uno inválido con código uno, que representa rechazo esperado y no una excepción.

Escena 12. Ejecución y escritorío

08:00–08:40

Responsable: Hugo

En pantalla: Mostrar `npm run dev`, la aplicación web y, sin compilar durante el video, el ejecutable o la configuración de Electron.

Hugo: Durante desarrollo se instala con `npm install` y se inicia con `npm run dev`. La versión de escritorio usa Electron y puede construirse como portable o como instalador NSIS para Windows x64. El ejecutable final incluye la aplicación, por lo que el usuario no necesita instalar Node, npm ni Java.

Escena 13. Comprobación de la rúbrica

08:40–09:15

Responsable: Ernesto y Hugo

En pantalla: Mostrar una tabla de ocho filas con los resultados verificados y una captura de un diagnóstico con ubicación.

Ernesto: Quedan demostrados los cuatro criterios de complejidad baja y los cuatro de complejidad media: válido, léxico, sintáctico y mixto.

Hugo: También se cumple la ubicación correcta por línea y columna, y el reporte inteligible en español con fase y símbolo o token relacionado. Así se cubren los diez criterios de evaluación con archivos reproducibles.

Escena 14. Conclusiones

09:15–09:40

Responsable: Ernesto y Hugo

En pantalla: Volver a la portada final con las palabras gramática, generación, recuperación y validación.

Ernesto: ANTLR permitió definir el lenguaje declarativamente y recuperar múltiples errores en una sola ejecución.

Hugo: La matriz y sus pruebas conectan cada requisito con evidencia concreta. El resultado es un analizador reutilizable, documentado y verificable. Muchas gracias.

4 Referencia para la demostración

4.1 Matriz oficial de la rúbrica

Hugo presenta los cuatro casos de `low`; Ernesto presenta los cuatro de `medium`.

Caso	Tokens	LEX	SYN
low/valid.cps	112	0	0
low/lexer_errors.cps	112	3	0
low/parser_errors.cps	109	0	3
low/lexer_parser_errors.cps	110	2	2
medium/valid.cps	263	0	0
medium/lexer_errors.cps	263	3	0
medium/parser_errors.cps	260	0	3
medium/lexer_parser_errors.cps	261	2	2

4.2 Correspondencia de complejidad

Nivel	Construcciones conservadas en sus cuatro archivos
Bajo	Variables <code>integer</code> , <code>string</code> y <code>boolean</code> ; constante; suma y multiplicación; <code>if</code> ; <code>while</code> ; <code>foreach</code> .
Medio	Todo lo anterior; arreglo declarado y usado; clases <code>Estudiante</code> y <code>Profesor</code> ; dos expresiones <code>new</code> ; funciones <code>sumar</code> y <code>mostrarCurso</code> ; dos llamadas.

4.3 Comandos rápidos

```
npm run cli -- examples/rubric/low/valid.cps
npm run cli -- examples/rubric/low/lexer_errors.cps
npm run cli -- examples/rubric/low/parser_errors.cps
npm run cli -- examples/rubric/low/lexer_parser_errors.cps
npm run cli -- examples/rubric/medium/valid.cps
npm run cli -- examples/rubric/medium/lexer_errors.cps
npm run cli -- examples/rubric/medium/parser_errors.cps
npm run cli -- examples/rubric/medium/lexer_parser_errors.cps
```

Los dos casos válidos terminan con código 0. Los seis casos incorrectos terminan con código 1 porque el rechazo es intencional.

4.4 Carpeta complementaria de recuperación

`examples/error_distribution` conserva tres archivos adicionales para estudiar agrupación, recuperación y alternancia prolongada:

Archivo	Tokens	LEX	SYN
<code>lexer_errors.cps</code>	39	3	0
<code>parser_errors.cps</code>	64	0	7
<code>lexer_parser_alternating_errors.cps</code>	72	4	4

Estos archivos son evidencia suplementaria; durante el video tienen prioridad los ocho casos que corresponden directamente a la rúbrica.

5 Lista final de verificación

- ☐ Duración ensayada menor o igual a 9:40.
- ☐ Hablan únicamente Hugo y Ernesto.
- ☐ Se muestran los ocho archivos de la matriz.
- ☐ Se ven los conteos 0/0, 3/0, 0/3 y 2/2 en ambos niveles.
- ☐ Se muestra línea, columna, fase, símbolo y mensaje en español.
- ☐ Se explica la diferencia entre lexer y parser.
- ☐ Se muestran tokens y árbol de parseo.
- ☐ Se ejecuta `npm test`: 3 archivos y 24 pruebas aprobadas.
- ☐ Se menciona el alcance sin semántica ni ejecución de Compiscript.
- ☐ El enlace final de YouTube se coloca en el README.

DragonsSlayers 2.0

Laboratorio 01 – Construcción de Compiladores CC-3032

Guion definitivo: 9 minutos 40 segundos.